

Relatórios de Redes Neurais

Grupo

- Claudio Caueh de Oliveira Xavier
- Lucas Mairon Oliveira Camilo

Este relatório descreve dois modelos de LLM: um voltado para a construção de um chatbot e outro para a classificação de sentimentos em frases. A seguir, cada um é detalhado separadamente.

Questão 7 — ChatBot

Modelo Utilizado

O modelo utilizado foi o **Nemotron-Labs-Diffusion-3B**, proposto pela NVIDIA e lançado em 19 de maio de 2026. O modelo conta com 3 bilhões de parâmetros e opera em três modos distintos: AR, DLM e SPEC.

Modo AR

Os tokens são processados sequencialmente, da esquerda para a direita, de forma semelhante ao que ocorre em uma transformer tradicional. Por não exigir tanto processamento paralelo, esse modo é mais adequado para servidores em nuvem com alta concorrência.

Modo DLM

Os tokens são processados em blocos, o que permite paralelismo. Para isso, o modelo recebe duas entradas distintas: uma com o texto original e outra com o texto mascarado e manipulado. Dessa forma, o modelo tenta prever o máximo possível a partir da entrada mascarada e consulta a entrada original quando necessário.

Modo SPEC

Representa uma combinação dos dois modos anteriores: utiliza a difusão para gerar os tokens candidatos e o modo AR para verificar a coerência das palavras e selecionar as mais adequadas ao contexto.

Código Fonte

```
!pip install -q "transformers>=5.0.0" accelerate
```

```
from transformers import AutoModel, AutoTokenizer
import torch
import time
```

Para o código, foi necessário instalar a biblioteca `transformers`, responsável por carregar e executar o modelo, e a `accelerate`, utilizada para distribuir os pesos do modelo entre CPU e GPU automaticamente. As importações incluem `AutoModel` e `AutoTokenizer` para carregar o modelo e seu tokenizador, `torch` para operações com tensores e execução na GPU, e `time` para medir o tempo de resposta em cada modo de inferência.

```
REPO = "nvidia/Nemotron-Labs-Diffusion-3B"
OUTPUT_DIR = "./nemotron_diffusion_lora"

print("Carregando tokenizer...")
tokenizer = AutoTokenizer.from_pretrained(REPO, trust_remote_code=True)
tokenizer.pad_token = tokenizer.eos_token
tokenizer.padding_side = "right"

print("Carregando modelo para treino...")
model = AutoModel.from_pretrained(
    REPO,
    trust_remote_code=True,
    device_map="auto",
    cache_dir="./cache",
)
model.config.use_cache = True
```

O trecho acima realiza o carregamento do modelo e do tokenizador a partir do repositório oficial, utilizando as funções de auto-identificação da biblioteca `transformers`. O argumento `device_map="auto"` delega à biblioteca `accelerate` a decisão de alocar cada camada do modelo nos dispositivos disponíveis.

```
FRASES = [
    "O que é inteligência artificial?",
    "Como funciona a fotossíntese?",
    "Explique o conceito de inflação de forma simples.",
    "Quais são as principais linguagens de programação?",
```

```

    "0 que é um buraco negro?",
]

SYSTEM_PROMPT = "Você é um assistente prestativo que responde em português."
MAX_NEW_TOKENS = 256
BLOCK_LENGTH = 32

if MAX_NEW_TOKENS % BLOCK_LENGTH != 0:
    MAX_NEW_TOKENS = (MAX_NEW_TOKENS // BLOCK_LENGTH) * BLOCK_LENGTH

```

```

def gerar(pergunta, mode):
    messages = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": pergunta},
    ]
    prompt = tokenizer.apply_chat_template(messages, tokenize=False,
add_generation_prompt=True)
    prompt_ids = tokenizer(prompt,
return_tensors="pt").input_ids.to(model.device)
    n_prompt_tokens = prompt_ids.shape[1]

    inicio = time.time()
    with torch.no_grad():
        if mode == "ar":
            out_ids, nfe = model.ar_generate(
                prompt_ids,
                max_new_tokens=MAX_NEW_TOKENS,
            )
        elif mode == "spec":
            out_ids, nfe = model.linear_spec_generate(
                prompt_ids,
                max_new_tokens=MAX_NEW_TOKENS,
                block_length=BLOCK_LENGTH,
                eos_token_id=tokenizer.eos_token_id,
            )
        else: # dlm
            out_ids, nfe = model.generate(
                prompt_ids,
                max_new_tokens=MAX_NEW_TOKENS,
                block_length=BLOCK_LENGTH,
                threshold=0.9,
                eos_token_id=tokenizer.eos_token_id,

```

```

    )
    tempo = time.time() - inicio

    n_tokens_gerados = out_ids.shape[1] - n_prompt_tokens
    resposta = tokenizer.batch_decode(
        out_ids[:, n_prompt_tokens:], skip_special_tokens=True
    )[0].strip()

    return {
        "resposta": resposta,
        "nfe": nfe,
        "tempo": round(tempo, 2),
        "tokens": n_tokens_gerados,
        "tok_seg": round(n_tokens_gerados / tempo, 1),
    }

```

```

model.eval()
model.config.use_cache = True

MODOS = ["ar", "dln", "spec"]
resultados = []

for frase in FRASES:
    print(f"\n{'='*70}")
    print(f"📝 Pergunta: {frase}")
    print(f"{'='*70}")
    linha = {"pergunta": frase}

    for modo in MODOS:
        print(f"\n ⌚ Gerando [{modo.upper()}]...")
        r = gerar(frase, modo)
        linha[modo] = r
        print(f" 🕒 [{modo.upper()}] {r['resposta'][:120]}{'...' if
len(r['resposta']) > 120 else ''}")
        print(f"      NFE={r['nfe']} | Tempo={r['tempo']}s | Tokens=
{r['tokens']} | {r['tok_seg']} tok/s")

    resultados.append(linha)

```

```

print(f"\n\n{'='*70}")
print(f"📊 RESUMO COMPARATIVO")
print(f"{'='*70}")

```

```

print(f"{'Pergunta':<35} {'Modo':<6} {'NFE':>6} {'Tempo':>7}
{'Tokens':>7} {'tok/s':>7}")
print("-"*70)

for linha in resultados:
    for i, modo in enumerate(MODOS):
        r = linha[modo]
        pergunta_str = linha['pergunta'][:33] + ".." if i == 0 else " " *
35
        print(f"{pergunta_str:<35} {modo.upper():<6} {r['nfe']:>6}
{r['tempo']:>6}s {r['tokens']:>7} {r['tok_seg']:>7}")
        print("-"*70)

# Médias por modo
print(f"\n{'Médias por modo':^70}")
print(f"{'Modo':<6} {'NFE médio':>10} {'Tempo médio':>12} {'Tokens
médio':>13} {'tok/s médio':>12}")
print("-"*55)
for modo in MODOS:
    nfes = [r[modo]['nfe'] for r in resultados]
    tempos = [r[modo]['tempo'] for r in resultados]
    tokens = [r[modo]['tokens'] for r in resultados]
    toks = [r[modo]['tok_seg'] for r in resultados]
    print(f"{modo.upper():<6} {sum(nfes)/len(nfes):>10.1f}
{sum(tempos)/len(tempos):>11.2f}s {sum(tokens)/len(tokens):>13.1f}
{sum(toks)/len(toks):>12.1f}")

```

O restante do código trata da avaliação comparativa do modelo, executando as cinco perguntas nos três modos disponíveis e exibindo as métricas de desempenho de cada combinação.

Com o modelo pronto, foi possível realizar o streamlit, para apresentar uma versão mais amigável ao cliente do código, estando disponível o código do mesmo abaixo:

```

import os
import gc
import time
import torch
from pathlib import Path
import streamlit as st
from transformers import AutoModel, AutoTokenizer, BitsAndBytesConfig
from peft import PeftModel

# O uso dessa variavel de ambiente foi porque o bitsandbytes não suporta

```

```

a versão
# mais recente do cuda no momento que esse código foi criado
os.environ["BNB_CUDA_VERSION"] = "130"

|
st.set_page_config(
    page_title="Nemotron Multi-Mode ChatBot",
    layout="wide",
    initial_sidebar_state="expanded"
)
|
st.markdown("""
<style>
    .main .block-container {
        padding-top: 2rem;
        padding-bottom: 2rem;
    }
    /* Reduzir o espaçamento do topo da barra lateral */
    [data-testid="stSidebar"] .block-container {
        padding-top: 1.5rem !important;
        padding-bottom: 1.5rem !important;
    }
    [data-testid="stSidebarHeader"] {
        display: none !important;
    }
    .metric-card {
        background-color: rgba(255, 255, 255, 0.05);
        border: 1px solid rgba(255, 255, 255, 0.1);
        border-radius: 8px;
        padding: 10px 15px;
        margin-top: 5px;
        margin-bottom: 10px;
    }
    .sidebar-label {
        font-size: 0.95rem;
        font-weight: 600;
        padding-top: 6px;
    }
    .stDeployButton {
        display: none !important;
    }
</style>
""", unsafe_allow_html=True)
|

```

```

if "messages" not in st.session_state:
    st.session_state.messages = []

if "current_model_config" not in st.session_state:
    st.session_state.current_model_config = {
        "repo_name": "",
        "quantization": "",
        "use_lora": False
    }

if "last_loaded_config" not in st.session_state:
    st.session_state.last_loaded_config = {
        "repo_name": "",
        "quantization": "",
        "use_lora": False
    }

if "model_unloaded" not in st.session_state:
    st.session_state.model_unloaded = False

# Essa limpeza é essencial para evitar erros de OOM (Out of Memory) na
# GPU ao alterar as configurações de execução do modelo
def cleanup_memory():
    model = st.session_state.get("model")
    tokenizer = st.session_state.get("tokenizer")

    if "model" in st.session_state:
        st.session_state.model = None
        del st.session_state.model
    if "tokenizer" in st.session_state:
        st.session_state.tokenizer = None
        del st.session_state.tokenizer

    for key in list(globals().keys()):
        if key in ["model", "tokenizer", "base_model"]:
            globals()[key] = None

    if model is not None:
        try:
            model.to("cpu")
        except Exception:
            pass
        del model

```

```

    if tokenizer is not None:
        del tokenizer

    gc.collect()
    if torch.cuda.is_available():
        torch.cuda.empty_cache()
        gc.collect()
        torch.cuda.empty_cache()
        torch.cuda.reset_peak_memory_stats()

    st.session_state.current_model_config = {
        "repo_name": "",
        "quantization": "",
        "use_lora": False
    }

def load_model(repo_name, quant_mode, use_lora):
    config = st.session_state.current_model_config

    if (config["repo_name"] == repo_name and
        config["quantization"] == quant_mode and
        config["use_lora"] == use_lora and
        "model" in st.session_state and
        "tokenizer" in st.session_state):
        return st.session_state.tokenizer, st.session_state.model

    if "model" in st.session_state or "tokenizer" in st.session_state:
        cleanup_memory()

    if repo_name == "nvidia/Nemotron-Labs-Diffusion-3B" and (Path.cwd() /
"hf_repo").exists():
        model_path = str(Path.cwd() / "hf_repo")
        is_local = True
    else:
        model_path = repo_name
        is_local = False

    try:
        tokenizer = AutoTokenizer.from_pretrained(model_path,
trust_remote_code=True)

        if quant_mode == "4-bit (NF4)":

```

```

        quantization_config = BitsAndBytesConfig(
            load_in_4bit=True,
            bnb_4bit_compute_dtype=torch.bfloat16,
            bnb_4bit_quant_type="nf4",
            bnb_4bit_use_double_quant=True
        )
        base_model = AutoModel.from_pretrained(
            model_path,
            trust_remote_code=True,
            quantization_config=quantization_config,
            device_map="auto"
        )
    elif quant_mode == "8-bit":
        quantization_config = BitsAndBytesConfig(
            load_in_8bit=True
        )
        base_model = AutoModel.from_pretrained(
            model_path,
            trust_remote_code=True,
            quantization_config=quantization_config,
            device_map="auto"
        )
    else:
        base_model = AutoModel.from_pretrained(
            model_path,
            trust_remote_code=True,
            torch_dtype=torch.bfloat16,
            device_map="auto"
        )

    if use_lora:
        lora_path = Path(model_path) / "linear_spec_lora"
        if lora_path.exists():
            model = PeftModel.from_pretrained(base_model,
str(lora_path)).eval().model
        elif not is_local and repo_name == "nvidia/Nemotron-Labs-
Diffusion-3B":
            model = PeftModel.from_pretrained(base_model, repo_name,
subfolder="linear_spec_lora").eval().model
        else:
            st.sidebar.warning("Adaptador LoRA não encontrado.
Carregando modelo base.")
            model = base_model

```

```

    else:
        model = base_model

    st.session_state.model = model
    st.session_state.tokenizer = tokenizer
    st.session_state.current_model_config = {
        "repo_name": repo_name,
        "quantization": quant_mode,
        "use_lora": use_lora
    }
    st.session_state.last_loaded_config = {
        "repo_name": repo_name,
        "quantization": quant_mode,
        "use_lora": use_lora
    }

    return tokenizer, model
except Exception as e:
    st.error(f"Erro ao carregar o modelo: {str(e)}")
    cleanup_memory()
    return None, None

# =====
# DESIGN DA INTERFACE E BARRA LATERAL
# =====
st.sidebar.title("Configurações do Nemotron")

if torch.cuda.is_available():
    allocated = torch.cuda.memory_allocated() / (1024 ** 3)
    reserved = torch.cuda.memory_reserved() / (1024 ** 3)
    peak = torch.cuda.max_memory_allocated() / (1024 ** 3)

    st.sidebar.markdown(f"""
    <h3 style="margin-bottom: 5px; margin-top: 0px; font-size:
    1.2rem;">Status da VRAM</h3>
    <div class="metric-card">
        <div><strong>Alocada:</strong> {allocated:.2f} GB</div>
        <div><strong>Reservada:</strong> {reserved:.2f} GB</div>
        <div><strong>Pico Alocado:</strong> {peak:.2f} GB</div>
    </div>
    """, unsafe_allow_html=True)
else:
    st.sidebar.markdown('<h3 style="margin-bottom: 5px; margin-top: 0px;

```

```

font-size: 1.2rem;">Status da VRAM</h3>', unsafe_allow_html=True)
    st.sidebar.warning("CUDA não disponível no PyTorch!")

# --- SELEÇÃO DO MODELO ---
st.sidebar.markdown("### Modelo e Precisão")
model_options = [
    "nvidia/Nemotron-Labs-Diffusion-3B",
    "nvidia/Nemotron-Labs-Diffusion-8B",
    "nvidia/Nemotron-Labs-Diffusion-14B",
    "Personalizado..."
]

col_mod1, col_mod2 = st.sidebar.columns([3, 7])
with col_mod1:
    st.markdown('<div class="sidebar-label">Modelo:</div>',
unsafe_allow_html=True)
with col_mod2:
    selected_option = st.selectbox("Modelo:", model_options, index=0,
label_visibility="collapsed")

if selected_option == "Personalizado...":
    repo_input = st.sidebar.text_input("Nome do Repositório do HF:",
value="nvidia/Nemotron-Labs-Diffusion-3B")
else:
    repo_input = selected_option

col_q1, col_q2 = st.sidebar.columns([3, 7])
with col_q1:
    st.markdown('<div class="sidebar-label">Quantização:</div>',
unsafe_allow_html=True)
with col_q2:
    quant_input = st.selectbox(
        "Quantização:",
        ["4-bit (NF4) [Recomendado]", "8-bit", "Precisão Total (BF16)"],
        index=0,
        label_visibility="collapsed"
    )
    quant_mode = "4-bit (NF4)" if "4-bit" in quant_input else ("8-bit" if "8-
bit" in quant_input else "BF16")

use_lora_input = st.sidebar.checkbox(
    "Ativar Adaptador LoRA Especulativo (se disponível)",
    value=True if repo_input == "nvidia/Nemotron-Labs-Diffusion-3B" else

```

```

False
)

last_cfg = st.session_state.last_loaded_config
if (last_cfg["repo_name"] != repo_input or
    last_cfg["quantization"] != quant_mode or
    last_cfg["use_lora"] != use_lora_input):
    st.session_state.model_unloaded = False

# MODO DE GERAÇÃO
st.sidebar.markdown("### Configurações de Inferência")
gen_mode = st.sidebar.selectbox(
    "Modo de Geração:",
    [
        "Linear Self-Speculation (Especulativo)",
        "AR (Autoregressivo Padrão)",
        "dLM (Diffusion Paralelo)"
    ],
    index=0
)

max_tokens = st.sidebar.slider("Máximo de novos tokens:", min_value=32,
max_value=1024, value=512, step=32)
block_len = st.sidebar.slider("Tamanho do Bloco (block_length):",
min_value=2, max_value=64, value=32, step=2)
threshold = st.sidebar.slider("Limiar (threshold):", min_value=0.0,
max_value=1.0, value=0.0, step=0.05)
temp = st.sidebar.slider("Temperatura:", min_value=0.0, max_value=1.5,
value=0.0, step=0.1)

if "dLM" in gen_mode:
    if max_tokens % block_len != 0:
        adjusted_tokens = (max_tokens // block_len) * block_len
        if adjusted_tokens == 0:
            adjusted_tokens = block_len
        st.sidebar.info(f"Ajustado 'Máximo de tokens' para
{adjusted_tokens} para ser múltiplo de {block_len} (exigido no modo
dLM).")
        max_tokens = adjusted_tokens

st.sidebar.markdown("### Ações")
col_clean1, col_clean2 = st.sidebar.columns(2)
with col_clean1:

```

```

    if st.button("Limpar Histórico", use_container_width=True):
        st.session_state.messages = []
        st.rerun()
with col_clean2:
    if st.session_state.model_unloaded:
        if st.button("Carregar Modelo", type="primary",
use_container_width=True):
            st.session_state.model_unloaded = False
            st.rerun()
        else:
            if st.button("Liberar VRAM", use_container_width=True):
                cleanup_memory()
                st.session_state.model_unloaded = True
                st.success("VRAM limpa!")
                time.sleep(1)
                st.rerun()
|
# =====
# CARREGAMENTO DO MODELO (SEÇÃO ATIVA)
# =====
if not st.session_state.model_unloaded:
    with st.spinner("Carregando o modelo e preparando pesos na GPU..."):
        tokenizer, model = load_model(repo_input, quant_mode,
use_lora_input)
else:
    tokenizer, model = None, None
|
# =====
# ÁREA PRINCIPAL DO CHAT
# =====
st.title("Nemotron Multi-Mode Chat")
st.markdown(f"**Modelo Ativo:** `{repo_input}` | **Precisão:**
`{quant_mode}` | **Modo de Geração:** `{gen_mode}`")
|
if model is None or tokenizer is None:
    st.warning("O modelo não está carregado. Clique em 'Carregar Modelo'
na barra lateral para iniciar.")
    st.stop()
|
for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])
        if "metrics" in message:

```

```

        st.caption(message["metrics"])
    |
    if user_input := st.chat_input("Digite sua mensagem..."):
        with st.chat_message("user"):
            st.markdown(user_input)
            st.session_state.messages.append({"role": "user", "content":
user_input})
        chat_history = [{"role": m["role"], "content": m["content"]} for m in
st.session_state.messages]
        try:
            prompt_formatted = tokenizer.apply_chat_template(chat_history,
tokenizer=False, add_generation_prompt=True)
            prompt_ids = tokenizer(prompt_formatted,
return_tensors='pt').input_ids.to('cuda')
        except Exception as e:
            st.error(f"Erro ao formatar histórico de mensagens: {str(e)}")
            st.stop()
        with st.chat_message("assistant"):
            with st.spinner("Nemotron respondendo..."):
                start_time = time.time()
                try:
                    if "AR" in gen_mode:
                        out_ids, nfe = model.ar_generate(
                            prompt_ids,
                            max_new_tokens=max_tokens,
                            temperature=temp,
                            eos_token_id=tokenizer.eos_token_id
                        )
                    elif "dLM" in gen_mode:
                        out_ids, nfe = model.generate(
                            prompt_ids,
                            max_new_tokens=max_tokens,
                            block_length=block_len,
                            threshold=threshold,
                            temperature=temp,
                            eos_token_id=tokenizer.eos_token_id
                        )
                    else:
                        out_ids, nfe = model.linear_spec_generate(

```

```

        prompt_ids,
        max_new_tokens=max_tokens,
        block_length=block_len,
        threshold=threshold,
        temperature=temp,
        eos_token_id=tokenizer.eos_token_id
    )

    end_time = time.time()

    new_tokens_ids = out_ids[0, prompt_ids.shape[1]:]
    response_text = tokenizer.decode(new_tokens_ids,
skip_special_tokens=True)

    duration = end_time - start_time
    num_tokens = len(new_tokens_ids)
    tks = num_tokens / duration if duration > 0 else 0
    metrics_text = f"{num_tokens} tokens | {duration:.2f}s |
{tks:.2f} TK/s | NFE={nfe}"

    st.markdown(response_text)
    st.caption(metrics_text)

    st.session_state.messages.append({
        "role": "assistant",
        "content": response_text,
        "metrics": metrics_text
    })

    except Exception as e:
        st.error(f"Erro na inferência: {str(e)}")
        if "out of memory" in str(e).lower():
            st.warning("Estouro de memória da GPU detectado!
Liberando VRAM automaticamente...")
            cleanup_memory()
            st.session_state.model_unloaded = True
            st.rerun()
        else:
            st.info("Caso ocorra estouro de memória (OOM), clique
no botão 'Liberar VRAM' na barra lateral.")

```

Execução do Código

Os resultados obtidos no Google Colab estão registrados nas imagens abaixo.

```

=====
🔍 Pergunta: O que é inteligência artificial?
=====

🕒 Gerando [AR]...
🤖 [AR] A **inteligência artificial (IA)** é um campo da ciência da computação que se concentra em criar sistemas capazes de rea...
NFE=256 | Tempo=12.28s | Tokens=256 | 20.8 tok/s

🕒 Gerando [DLM]...
🤖 [DLM] A **inteligência artificial** (IA)** é uma área da ciência da computação que busca ** criar sistemas capazes de realizar...
NFE=221 | Tempo=48.68s | Tokens=256 | 5.3 tok/s

🕒 Gerando [SPEC]...
🤖 [SPEC] A **inteligência artificial (IA)** é um campo da ciência da computação que se concentra em criar sistemas capazes de rea..
NFE=185 | Tempo=44.5s | Tokens=257 | 5.8 tok/s

=====
🔍 Pergunta: Como funciona a fotossíntese?
=====

🕒 Gerando [AR]...
🤖 [AR] A **fotossíntese** é o processo pelo qual as plantas, algas e algumas bactérias (como cianobactérias) convertem a energi...
NFE=256 | Tempo=10.97s | Tokens=256 | 23.3 tok/s

🕒 Gerando [DLM]...
🤖 [DLM] A fototossíntese** é um processo fundamental pelo qual as plantas, algas e algumas bactérias convertem a luz solar em en...
NFE=219 | Tempo=51.83s | Tokens=256 | 4.9 tok/s

🕒 Gerando [SPEC]...
🤖 [SPEC] A **fotossíntese** é o processo pelo qual as plantas, algas e algumas bactérias (como cianobactérias) convertem a luz so..
NFE=169 | Tempo=40.6s | Tokens=256 | 6.3 tok/s

=====
🔍 Pergunta: Explique o conceito de inflação de forma simples.
=====

```

RESUMO COMPARATIVO					
Pergunta	Modo	NFE	Tempo	Tokens	tok/s
O que é inteligência artificial?..	AR	256	12.28s	256	20.8
	DLM	221	48.68s	256	5.3
	SPEC	185	44.5s	257	5.8
Como funciona a fotossíntese?..	AR	256	10.97s	256	23.3
	DLM	219	51.83s	256	4.9
	SPEC	169	40.6s	256	6.3
Explique o conceito de inflação d..	AR	256	10.9s	256	23.5
	DLM	234	55.86s	256	4.6
	SPEC	171	40.9s	258	6.3
Quais são as principais linguagen..	AR	256	10.9s	256	23.5
	DLM	197	47.19s	256	5.4
	SPEC	143	34.12s	257	7.5
O que é um buraco negro?..	AR	256	11.41s	256	22.4
	DLM	219	53.51s	256	4.8
	SPEC	189	45.34s	256	5.6
Médias por modo					
Modo	NFE médio	Tempo médio	Tokens médio	tok/s médio	
AR	256.0	11.29s	256.0	22.7	
DLM	218.0	51.41s	256.0	5.0	
SPEC	171.4	41.09s	256.8	6.3	

Análise dos Resultados

Com base nos resultados obtidos, é possível observar que:

- O modo AR apresentou o melhor desempenho em velocidade, o que provavelmente se deve às limitações de hardware do ambiente Colab, onde o processamento paralelo do modo DLM não encontra tanto benefício.
- Os três modos produziram respostas adequadas e semanticamente semelhantes entre si, indicando que todos são funcionalmente equivalentes para os fins propostos neste experimento.

Questão 8 — Análise de Sentimentos

Resumo Executivo

A seguir está a descrição do desenvolvimento, a avaliação de desempenho e a análise de limitações de um classificador de sentimentos voltado para a língua portuguesa (tweets). O sistema categoriza sentenças em três classes mutuamente exclusivas: **positivo**, **negativo** e **neutro**. O comportamento do modelo de linguagem pré-treinado **Qwen 3.5 0.8B Instruct** em sua versão original foi comparado com o da versão ajustada via Fine-Tuning Supervisionado (SFT) direto, sem etapas intermediárias de raciocínio.

1. Pipeline do Sistema

O pipeline de processamento e classificação foi implementado em Python com a biblioteca `transformers` da Hugging Face, integrada ao framework **Unsloth** para fine-tuning otimizado por hardware. O fluxo de inferência em lote para classificação segue as etapas abaixo:



Etapas Detalhadas

- Tokenização e Padding:** o texto bruto é convertido em uma sequência de IDs numéricos pelo tokenizador. Para inferência em lote, o padding é configurado à esquerda (`padding_side = 'left'`), utilizando o `eos_token` como caractere de preenchimento, o que garante o alinhamento correto das saídas do modelo causal.
- Formatação de Prompt (Chat Template):** a entrada é formatada no padrão ChatML via `apply_chat_template` , com os seguintes componentes:

- **System Prompt:** *"Você é um classificador de análise de sentimentos em português. Analise o texto do usuário e responda com o rótulo final (positivo, negativo ou neutro)."*
 - **User Prompt:** *"Classifique o sentimento deste texto: \"{TEXTO}\""*
 - **Desativação do raciocínio interno:** o modelo Qwen 3.5 possui suporte nativo a thinking steps. Para forçar uma resposta direta e reduzir latência, o template foi configurado com `enable_thinking=False` (ou equivalentemente fechando o bloco `<think>\n\n</think>\n\n` no prompt), fazendo o modelo gerar apenas o rótulo de classificação.
3. **Geração:** os tensores de entrada são enviados para a GPU e o modelo gera o mínimo de tokens necessário (`max_new_tokens=15`).
 4. **Decodificação e Pós-processamento:** o texto gerado é decodificado, limpo de tokens especiais (como `<|im_end|>` e `<|endoftext|>`), convertido para caixa baixa e mapeado para um dos três rótulos oficiais: `positivo` , `negativo` ou `neutro` .
-

2. Modelo Utilizado e Processo de Treinamento

O modelo selecionado foi o **Qwen 3.5 0.8B Instruct** (`unsloth/Qwen3.5-0.8B`).

Justificativa da Escolha

- **Eficiência computacional:** com 800 milhões de parâmetros, o modelo executa inferências em milissegundos e consome menos de 2 GB de VRAM em precisão `FP16 / BF16` .
- **Desempenho em português:** a família Qwen 3.5 demonstra boa compreensão sintática, morfológica e semântica do português, incluindo gírias e expressões típicas de redes sociais.
- **Compatibilidade com LoRA:** o modelo aceita a fusão com adaptadores LoRA de forma estável, viabilizando o fine-tuning via Unsloth com recursos de hardware limitados.

Variantes Comparadas

- **Modelo Base Original:** versão de fábrica, avaliada diretamente via engenharia de prompts.
- **Modelo SFT Raw 10k:** versão ajustada por 1 época sobre 10.000 amostras de tweets em português balanceados (`train_raw_10k.csv`), treinada para classificação direta sem etapas intermediárias de raciocínio.

Origem e Curação dos Dados de Treino

O conjunto de dados final (`train_raw_10k.csv` , 10.000 amostras) foi construído a partir da mesclagem, limpeza e balanceamento de três fontes públicas do Kaggle:

1. [tweetpt](#) (`Train3Classes.csv`): postagens reais do Twitter em português, com sentimentos mapeados para 0 (negativo), 1 (positivo) e 2 (neutro).
2. [dataset-para-anlise-de-sentimentos-gemini](#) (`gemini.csv`): dataset sintético gerado via API do Gemini, com sentenças curtas em português e suas respectivas polaridades.
3. [anlise-de-sentimentos-pt-br](#) (`sentiment_analysis_pt_br.csv`): dataset generalista de análise de sentimentos adaptado para o português.

Processo de Tratamento de Dados

- **Correção de codificação:** aplicou-se limpeza para corrigir textos com codificação corrompida (mojibake), preservando acentuações, pontuações e emojis relevantes para a interpretação emocional.
- **Prevenção de vazamento de dados:** removeu-se qualquer sobreposição exata entre o conjunto de treino e o de teste final (`teste_final.csv`), preservando a integridade das métricas.
- **Balanceamento:** o dataset foi balanceado por subamostragem estratificada, resultando em exatamente 10.000 amostras — 3.333 positivas, 3.333 negativas e 3.334 neutras.

3. Análise dos Resultados

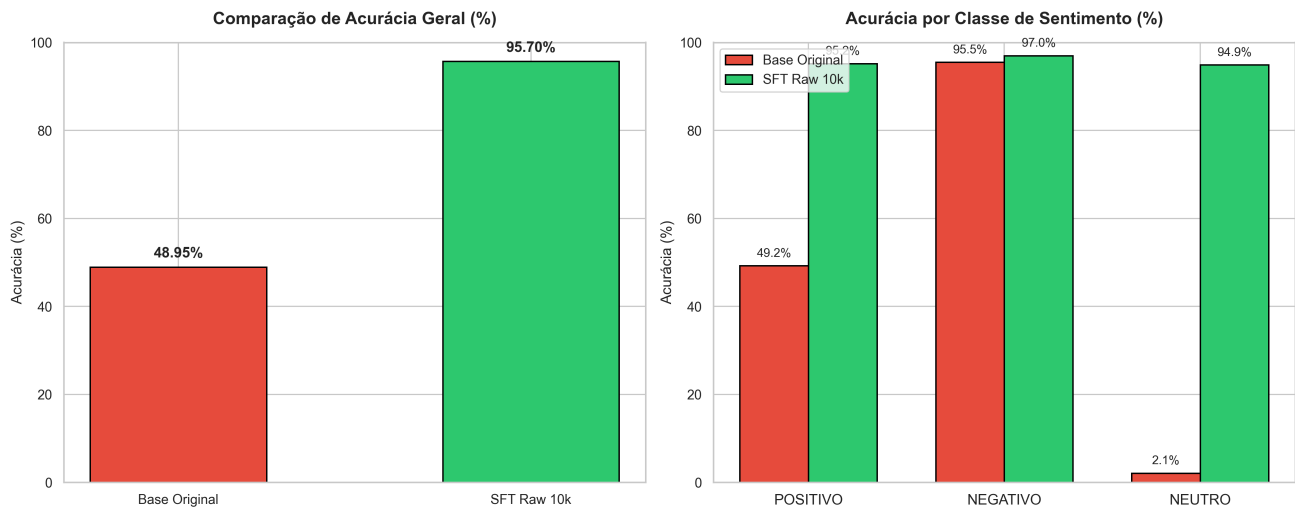
Os modelos foram avaliados em lote em dois conjuntos de teste distintos.

A. Conjunto de Teste Local Balanceado (999 amostras)

Amostra equilibrada extraída de `teste_fewShot.csv` (333 amostras por classe).

Modelo / Métrica	Acurácia Geral	Acurácia POSITIVO	Acurácia NEGATIVO	Acurácia NEUTRO
Base Original (Qwen 3.5)	48,95%	49,25%	95,50%	2,10%
SFT Raw 10k (Ajustado)	95,70%	95,20%	97,00%	94,89%
Diferença	+46,75 p.p.	+45,95 p.p.	+1,50 p.p.	+92,79 p.p.

O modelo base apresentou falha quase total na classe neutra, classificando a grande maioria dos textos informativos como "negativo" — indicando um viés severo de distribuição. O modelo ajustado atingiu 95,70% de acurácia geral, com desempenho equilibrado entre as três classes e tempo de inferência em lote de aproximadamente 0,02 segundos por tweet.

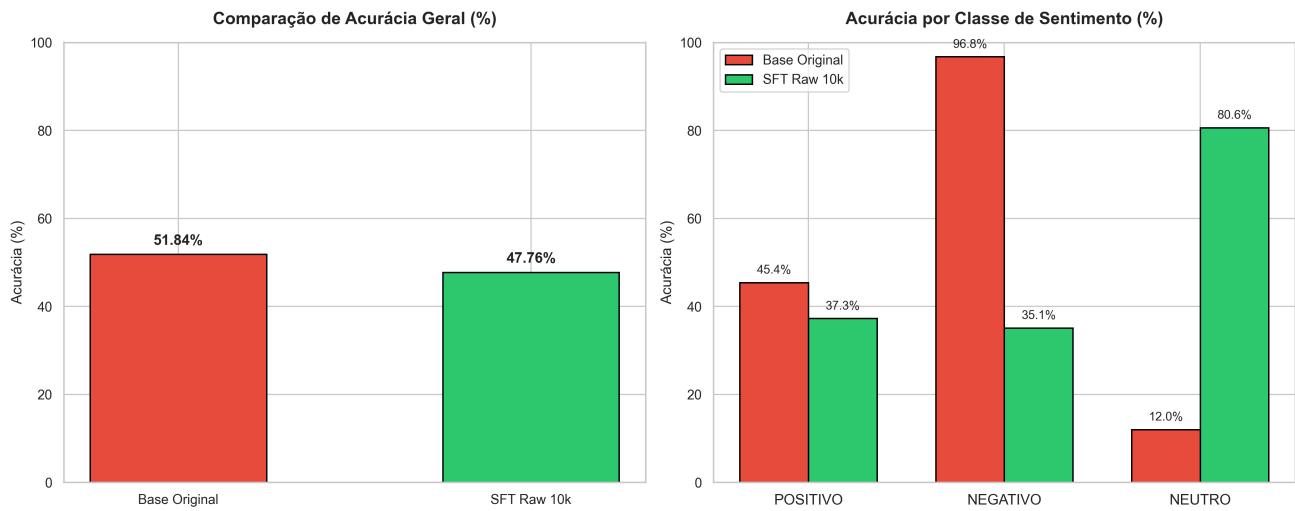


B. Conjunto de Teste Externo (tweetsentbr_fewshot — 2.010 amostras)

Dataset público externo, utilizado para avaliar o impacto do desvio de distribuição de domínio.

Modelo / Métrica	Acurácia Geral	Acurácia POSITIVO	Acurácia NEGATIVO	Acurácia NEUTRO
Base Original (Qwen 3.5)	51,84%	45,42%	96,76%	12,02%
SFT Raw 10k (Ajustado)	47,76%	37,27%	35,09%	80,62%
Diferença	-4,08 p.p.	-8,16 p.p.	-61,67 p.p.	+68,60 p.p.

O modelo ajustado teve queda de desempenho neste dataset externo. A causa identificada foi o uso de marcadores de anonimização sintéticos (`USERNAME` e `NUMBER`) no lugar de menções e números reais — um padrão ausente nos dados de treino. Esse desvio léxico fez o modelo tender fortemente para a classe neutra, com colapso nas demais.



4. Limitações do Modelo

- Sensibilidade ao desvio de domínio:** a acurácia do modelo ajustado depende do formato léxico típico do Twitter. A presença de marcadores sintéticos ou vocabulário de outros domínios (como notícias ou textos corporativos) leva o modelo a classificar a maioria dos textos como neutro.
- Dependência do chat template:** o modelo base requer um prompt estruturado corretamente. Sem o fechamento adequado das tags `<think>`, ele pode gerar raciocínio intermediário ou saídas redundantes dentro do limite curto de tokens configurado.
- Viés do modelo base em português:** sem fine-tuning, o modelo pré-treinado apresenta forte tendência a classificar textos como "negativo", independentemente do conteúdo. O ajuste supervisionado é necessário para alinhar o comportamento do modelo à tarefa de classificação.